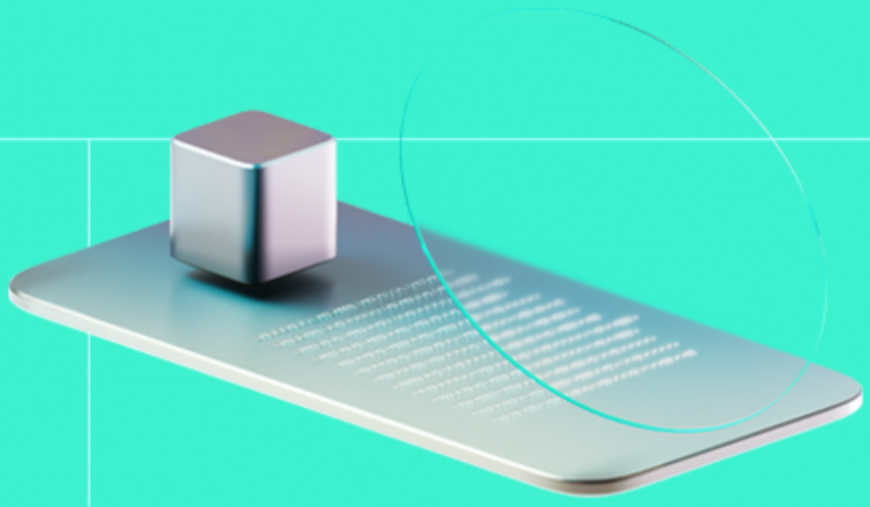
HACKEN

Smart Contract Code Review And Security Analysis Report



Customer: Tangible – USDR

Date: 09/09/2026

We express our gratitude to the Tangible – USDR team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

USDR Redemption v2 is a pro-rata redemption contract. Holders present USDR, which is burned, and receive a share of the USDC that has been funded, independent of redemption order. The audit covered the **USDRRedemption** implementation.

Document

Name	Smart Contract Code Review and Security Analysis Report for Tangible – USDR
Audited By	Kerem Solmaz
Approved By	Olesia Bilenka
Website	https://www.tangible.store/
Changelog	08/09/2026 - Preliminary Report
	09/09/2026 - Final Report
Platform	Polygon
Language	Solidity
Tags	Fungible Token, Decentralized Finance (DeFi), Centralization
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/TangibleTNFT/usdr-redemption-v2
Initial Commit	e6f27d4
Final Commit	62d30b0

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

3	3	0	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	0
Low	1

Vulnerability	Severity	Status
F-2026-19264 - Uncapped Rate Increase Clears The Sweep Clock And Risks Locking Residual USDC	Low	Fixed
F-2026-19265 - Funding Target Uses Configured Supply And Leaves Surplus Idle Until Sweep	Info	Fixed
F-2026-19266 - Missing Self-Address Check In Redeem And Claim	Info	Fixed

Documentation quality

- The repository includes a detailed README that states the pro-rata model, unit conventions, deploy parameters, and wind-down rules.
- Public functions on **USDRRedemption** and **IUSDRRedemption** carry extensive NatSpec.
- Documentation and implementation agree on share registration at redeem time, funding-based payouts, the 180-day post-full-funding sweep, the one-dollar rate ceiling, and that the sweep countdown arms only when remaining funding reaches zero, including the configured-supply slice that cannot be presented on Polygon.
- Some operational assumptions, including that the owner is a Gnosis Safe, are documented rather than enforced on-chain.

Code quality

- The implementation is a single non-upgradeable Solidity 0.8.30 contract that uses OpenZeppelin Contracts v5.6.1 for two-step ownership, transient reentrancy protection, SafeERC20 transfers, and mulDiv arithmetic.

Test coverage

- Line, statement, branch, and function coverage on **USDRRedemption** stands at 100% as measured by the Foundry coverage tool on the unit, fuzz, and invariant suites. The suite covers primary redemption, funding, rate, sweep, and rescue flows, plus fuzzed rounding properties for path, tranche, and order independence, and an invariant suite for solvency and the pro-rata ledger. Negative cases cover a rate above the on-chain ceiling, including a mistyped rate, and a USDC recipient equal to the redemption contract. The invariant suite asserts the rate ceiling. Polygon fork tests against the live USDR token exist but require an archive RPC and are excluded from continuous integration. Continuous integration runs format, build, and non-fork tests.

Table of Contents

System Overview **6**

Privileged Roles: 6

Potential Risks **7**

Findings **9**

Vulnerability Details 9

Disclaimers 15

Appendix 1. Definitions **16**

Severities 16

Potential Risks 16

Appendix 2. Scope **17**

Appendix 3. Additional Valuables **18**

System Overview

USDRRedemption is an immutable Polygon contract that converts USDR (ERC-20, 9 decimals) into native USDC (ERC-20, 6 decimals) on a pro-rata basis. The design replaces a first-come-first-served swap with a share ledger so that the order of redemptions does not change a holder's claim on funded USDC. The system consists of **USDRRedemption**, the **IUSDRRedemption** public interface, and the **IUSDR** burn surface used to destroy presented USDR. The USDR and USDC addresses and the redeemable population `totalUSDRSupply` are constructor immutables. That population is not read from the token total supply. The constructor requires USDR to report 9 decimals and USDC to report 6 decimals. The target rate is stored as USDC raw units per one whole USDR. It may only increase and cannot exceed 1,000,000 raw units, which is 1.00 USDC per whole USDR.

USDC enters the pro-rata ledger only when the owner funds the contract or recognizes an unaccounted balance already held. Funding cannot exceed the ceiling of supply times rate, rounded up, divided by 1e9. A holder who calls `redeem` burns USDR from themselves through **IUSDR**, registers matching shares, and is paid the current floor of shares times funded USDC divided by the immutable supply. Later funding is collected with `claim`. The rate does not appear in the payout formula. A packed position stores shares and USDC already paid. Raw USDC transfers are not funding unless the owner recognizes them. Redeem and claim reject the redemption contract itself as the USDC recipient.

When funded USDC reaches the ceiling, a timestamp is stored. After 180 days the owner may sweep the entire USDC balance, which permanently closes the contract. Holders who never presented USDR, and holders who presented it but never claimed the remainder, forfeit what is left. Full funding includes the configured-supply slice that may never be presented. That slice sits idle until the terminal sweep. If remaining funding never reaches zero, sweep stays locked. A rate increase clears the sweep clock unless the new ceiling is already met. While closed, redeem, claim, fund, and rate changes revert, and entitlement views return zero. Sweep remains callable so USDC received after closure can still be recovered. Stray non-USDC tokens can be rescued. USDC cannot.

Privileged Roles:

- Owner (**USDRRedemption**, inherited from OpenZeppelin Ownable2Step): Funds USDC, recognizes unaccounted USDC, raises the rate up to the on-chain ceiling, sweeps after the delay, rescues non-USDC tokens, and starts a two-step ownership transfer. The constructor accepts any non-zero address. Production comments expect a Gnosis Safe, but the contract does not enforce one. Compromise or misuse of this role allows an immediate rate increase that can clear the sweep clock, funding recognition, token rescue, and a sweep of remaining USDC once the delay has elapsed. Loss of the key before handover can freeze further funding and keep sweep locked if full funding is never reached. Ownership renounce is disabled.
- Pending owner (**USDRRedemption**, inherited from OpenZeppelin Ownable2Step): Completes ownership handover through `acceptOwnership()`. Until that call succeeds, the current owner retains every privileged path.
- Redeem and claim, including the overloads that choose a USDC receiver, may be called by any account.

Potential Risks

- **Scope Definition and Security Guarantees**: The audit covers **USDRRedemption**, **IUSDRRedemption**, and **IUSDR**. Tests, mocks, the Polygon deploy script, and the live USDR and native USDC token implementations are outside the review. Flaws in those out-of-scope components may still affect the audited contract because redemption, funding, and payouts call those tokens.
- **Dependency on External Logic for Implemented Logic**: Redemption burns the caller's USDR through the custom allowance-based burn on the out-of-scope USDR token. **USDRRedemption** never holds USDR and is not granted a burner role. If that burn reverts, spends allowance differently than **IUSDR** describes, or is blocked while the token is paused, new shares cannot be registered. Already registered positions can still be settled because claim does not call USDR.
- **System Reliance on External Contracts**: Redeem, claim, fund, sweep, stray-token recovery, and available-balance reads depend on the immutable USDR and USDC addresses set in **USDRRedemption**. Both tokens are upgradeable proxies. A proxy upgrade, a pause, or a change in transfer or burn behaviour can block burns or USDC movement while registered positions remain unpaid. The pro-rata denominator is an off-chain accessible-supply figure supplied at construction and is not read from the USDR token total supply. If that figure is below the USDR that can actually be presented, later holders cannot register shares. If it is above the presentable population, unused capacity leaves funded USDC unclaimable until a terminal sweep, or indefinitely if full funding never occurs.
- **Dependency on Unaudited External Libraries**: The project depends on OpenZeppelin Contracts v5.6.1 for two-step ownership, transient reentrancy protection, SafeERC20 transfers, and mulDiv arithmetic. While OpenZeppelin is widely used and has undergone prior audits, the specific version integrated has not been independently verified as part of this engagement. Any defect in those base contracts would affect **USDRRedemption**.
- **Owner's Unrestricted State Modification:** The owner may raise the rate on **USDRRedemption** at any time while the contract is open. The rate must be strictly greater than the current value and must not exceed 1,000,000 raw USDC units per whole USDR. A raise increases the funding target and clears the sweep clock unless the new ceiling is already met. There is no on-chain path to lower the rate. A raise to the ceiling after a full fund at the deploy rate leaves a remaining top-up on the same scale as the original funding target. Residual USDC still cannot leave through the stray-token recovery path. Existing positions remain entitled to a share of already recognized funding.
- **Coarse-grained Authorization Model Risks**: A single owner on **USDRRedemption** controls funding, rate increases, the terminal sweep, recovery of tokens other than USDC, and two-step ownership transfer. No separate roles isolate those powers. Compromise of that one address grants every privileged path at once.
- **Absence of Time-lock Mechanisms for Critical Operations**: Funding, rate increases, and non-USDC rescues on **USDRRedemption** execute in the calling transaction. The only delayed owner action is the terminal sweep, which waits 180 days after recognized funding equals the supply-times-rate ceiling.
- **Insufficient Multi-signature Controls for Critical Functions**: Ownership is a single address assigned in the **USDRRedemption** constructor. Production comments expect a Gnosis Safe, but the contract does not require one. Rate changes, funding, sweep, and stray-token recovery do not require a second on-chain signature.

- **Single Points of Failure and Control**: Funding progress, the rate, terminal closure, and stray-token recovery all require the current owner of **USDRRedemption**. Ownership cannot be renounced. If the key is lost before a two-step handover completes, further funding and rate increases stop, and the sweep may stay locked if full funding is never reached.
- **Administrative Key Control Risks**: After the sweep delay the owner of **USDRRedemption** sends the entire USDC balance to a chosen recipient and permanently closes the contract. Holders who never presented USDR, and holders who presented it but never claimed the remainder, forfeit what is left. Non-USDC tokens can be redirected immediately. Payouts follow the funded ledger, not the raw USDC balance, so holders who burn USDR before funding arrives receive a zero or partial payout and must wait on later claims. Unrecognized raw USDC transfers are not owed to anyone.

Findings

Vulnerability Details

F-2026-19264 - Uncapped Rate Increase Clears The Sweep Clock And Risks Locking Residual USDC - Low

Description:

The `setRate` public function accepts any strictly larger `rate` whose `expectedFunding` fits in `uint128`. Using the documented constructor value of 35,909,552 USDR, that ceiling is about `9.5e30`, so a mistyped value such as `533_000_000` (533 USDC per USDR) is accepted in place of the intended `533_000` (0.533 USDC per USDR).

`expectedFunding` is `ceil(totalUSDRSupply * rate / 1e9)`. The sweep clock `fullyFundedAt` is written only when `totalFunded` equals that target. `setRate` zeroes `fullyFundedAt` when the equality no longer holds. No other path arms the clock.

After that call the figures below apply. The large remainder is the extra USDC the owner must `fund` to restore equality. It is not the balance already on the contract.

Quantity	USDC (6 decimals)
Already funded (`totalFunded`)	19,103,881.664
New `expectedFunding` at `533_000_000`	19,139,791,216
`remainingFunding` (deposit required to re-arm)	19,120,687,334.336

Holders can still `redeem` and `claim` their share of the funded amount, `floor(shares * totalFunded / totalUSDRSupply)`. The leftover after those payouts has no owner exit. `setRate` cannot decrease. `rescueERC20` reverts `CannotRescueUSDC`. `sweep` reverts `SweepLocked` while `fullyFundedAt` is zero, at any later timestamp. The contract is not upgradeable. Residual USDC, including unclaimed positions and any surplus funded against the full supply, stays locked.

Assets:

- src/USDRRedemption.sol [https://github.com/TangibleTNFT/usdr-redemption-v2.git]

Status: Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	2/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

- Cap the increase allowed in a single `setRate` call so a mistyped argument cannot move `expectedFunding` by orders of magnitude.
- Optionally, add a way to reverse an unwanted raise (a decrease, or a propose and confirm flow) so `expectedFunding` can return to `totalFunded` and the clock can arm again.

Resolution: Fixed in commit `62d30b0`.

Added a hard cap `MAX_RATE = 1_000_000`, enforced in the `constructor` and `setRate` via `RateExceedsMax`. The documented mistype `533_000_000` now reverts, and `expectedFunding` at the cap stays in the same economic range as the original funding target.

F-2026-19265 - Funding Target Uses Configured Supply And Leaves Surplus Idle Until Sweep - Info

Description:

The `expectedFunding` public function is `ceil(totalUSDRSupply * rate / 1e9)`. `totalUSDRSupply` is an immutable constructor argument. It is not the USDR token `totalSupply`. The deployment script and NatSpec document an intended value of 35,909,552 USDR. Review-time presentable Polygon supply is about 34,002,333 USDR. The `_recordFunding` function arms `fullyFundedAt` only when `totalFunded` equals the configured-supply target.

Quantity	USDC (6 decimals)
`expectedFunding` at the configured supply	19,103,881.664
Review-time presentable supply times rate	18,089,241.156
Gap	1,014,640.508

If the owner deposits only 18,089,241.156 USDC, enough to pay the review-time presentable USDR at the current rate, the clock stays unset and `sweep` reverts `SweepLocked`. That state is not terminal. `remainingFunding` is still 1,014,640.508 USDC, and `fund` still accepts it. After that deposit the clock arms.

If the owner deposits the full target, the clock arms. Holders who present the review-time presentable supply can be paid 18,089,241.156 USDC. The remaining 1,014,640.508 USDC stays unclaimable unless more of the configured supply is later presented. It is locked until `fullyFundedAt` plus 180 days, then `sweep` returns it. `rescueERC20` rejects USDC on both paths. The same configured supply is the share cap on `redeem`.

Assets:

- src/USDRRedemption.sol [https://github.com/TangibleTNFT/usdr-redemption-v2.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

- Set `totalUSDRSupply` to the on-chain USDR supply.
- State in the documentation that `remainingFunding` must be deposited in full to arm `sweep`, including the slice that current holders cannot claim.

Resolution: Fixed in commit `62d30b0`.

The configured-supply denominator is unchanged. `README` and the fund `NatSpec` now state that the sweep countdown arms only when `remainingFunding()` is zero, including the unpresentable configured-supply slice, and that that slice returns through the terminal sweep.

F-2026-19266 - Missing Self-Address Check In Redeem And Claim - Info

Description:

The `redeem` and `claim` external functions that accept a `receiver`, pay settled USDC to that address and replace a zero `receiver` with `msg.sender`. Neither path rejects `receiver` equal to the contract.

A third-party `receiver` and a contract `receiver` both burn the caller's USDR, raise `paid` to full entitlement, emit a non-zero `Redeemed` payout, and leave `claimableUSDC` at 0 so `claim` reverts `NothingToClaim`. An arbitrary payee is the documented feature. A third-party destination leaves the contract and keeps `balance == totalFunded - totalPaid`. That case is not a defect.

A contract `receiver` leaves `availableUSDC` unchanged while `_settle` still increases `totalPaid`. Unaccounted surplus equals the payout: `balance - (totalFunded - totalPaid)`. `fundFromBalance` treats that gap as money that arrived outside `fund` and can `_recordFunding` it. The owner increases `totalFunded` by the settled amount without transferring USDC, so recorded funding exceeds deposits and `remainingFunding` falls by the payout.

Receiver	Holder USDC	Surplus	`remainingFunding` after `fundFromBalance`
Caller	Paid	0	Unchanged
Contract	0	Payout	Reduced by the payout
Third party	0	0	Unchanged

At half funding, redeeming `1_000_000e9` USDR to the contract reports `266_000e6` paid. `fundFromBalance` of that amount does not change the owner USDC balance and reduces `remainingFunding` by `266_000e6`.

Assets:

- src/USDRRedemption.sol [https://github.com/TangibleTNFT/usdr-redemption-v2.git]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Reject `receiver` equal to the contract in both functions.

Resolution: Fixed in commit `62d30b0`.

`redeem` and `claim` now reject receiver equal to the contract with `InvalidReceiver` on both internal payout helpers, so a settled transfer cannot park USDC as unaccounted surplus that `fundFromBalance` would recognize.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

hknio/severity-formula

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/TangibleTNFT/usdr-redemption-v2
Initial Commit	e6f27d4fe2fae3ee2325a8f966f31eeccea70ca2
Final Commit	62d30b0fa52793947171fecee1a7f0a48a3f0214
Whitepaper	-
Requirements	README.md
Technical Requirements	README.md

Asset	Type
src/USDRRedemption.sol [https://github.com/TangibleTNFT/usdr-redemption-v2.git]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment, and the Penetration Testing Execution Standard (PTES), These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.